SkillNavora

MODULE 1 HANDBOOK

Module 1: Programming and Software Engineering Foundations

7 sections, every topic explained with how it works, examples and resources.

From Non-Coder to Enterprise AI Builder: Learn, Build, Deploy, Get Hired and Launch Your AI Business.

Varun Kumar · SkillNavora · skillnavora.in

Edition: November 2026. Links point to official documentation where available; check them for the latest versions.
Financial figures are hypothetical examples, not advice.

Module 1: Programming and Software Engineering Foundations

Every section below explains the concept, how it works, each listed topic in detail, a worked example and where to learn more.

Sections

1. 1.1 Introduction to Programming
2. 1.2 Python Fundamentals
3. 1.3 JavaScript and TypeScript
4. 1.4 Data Structures and Algorithms
5. 1.5 Git and GitHub
6. 1.6 Software Engineering
7. 1.7 Development Tools

MODULE 1 · SECTION 1.1

1.1 Introduction to Programming

DEFINITION

> Programming is writing precise instructions that a computer can carry out. Software is the set of those instructions together with the data they use.

HOW IT WORKS

- A computer runs machine instructions. A programming language lets you write them in a form people can read, and a compiler or interpreter translates them.
- Compiled languages (for example C, Go) are translated ahead of time into machine code. Interpreted languages (for example Python, JavaScript) are read and run line by line by an interpreter or runtime.
- Frontend code runs in the user's browser or app. Backend code runs on a server and handles data, rules and integrations. They talk over HTTP, usually with JSON.
- An IDE (VS Code, Cursor) gives you an editor, a terminal, a debugger and version control in one place.

EVERY TOPIC EXPLAINED

What is programming and what is software. Programming is giving a computer exact, ordered instructions. Software is the finished set of those instructions plus the data and settings it uses, for example a mobile banking app, a spreadsheet or a website.

How computers execute instructions. The CPU fetches an instruction from memory, decodes it, executes it (add, compare, move data, jump) and moves to the next one, billions of times a second. Memory (RAM) holds running programs and data; storage (SSD) keeps files when power is off; the operating system shares the CPU, memory and devices between programs.

Programming languages; compiled vs interpreted. A programming language is a human-readable way to write instructions. A compiler (C, Go, Rust) translates the whole program into machine code before it runs, which is fast at run time. An interpreter (Python, JavaScript) reads and runs the code as it goes, which is quicker to write and test. Many modern languages mix both (Java and C# compile to bytecode run by a virtual machine).

Frontend vs backend. The frontend is what users see and click: web pages or app screens built with HTML, CSS and JavaScript. The backend runs on servers: it checks rules, stores and reads data, calls other services and returns results. An AI feature usually has a frontend chat box and a backend that calls the model API and the database.

Development environments and IDEs. An IDE (integrated development environment) such as VS Code or Cursor combines a code editor with syntax colouring, auto-complete, a terminal, a debugger, Git integration and extensions. A development environment also includes the installed language (Python), package manager (pip, npm) and project settings.

How applications are created and how software communicates. Applications are created by writing code, testing it, packaging it and deploying it to a computer or server. Software communicates over networks using protocols: HTTP for web requests, JSON as the data format, APIs as the agreed contract, and webhooks or queues for events. Example: your app sends an HTTP POST with JSON to an AI API and receives a JSON reply.

WORKED EXAMPLE

> A customer form on a website (frontend) sends a request to a server (backend), which saves the record in a database and replies with a confirmation.

RESOURCES

- Python tutorial (official) https://docs.python.org/3/tutorial/
- MDN: How the web works https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Web_standards/How_the_Web_works

1.2 Python Fundamentals

DEFINITION

> Python is a readable, general-purpose language widely used for data, automation, web backends and AI. Its basics are variables, types, control flow, functions and modules.

HOW IT WORKS

- Variables are names that point to values. Python infers the type (int, float, str, bool) from the value.
- Collections: lists (ordered, changeable), tuples (ordered, fixed), sets (unique items), dicts (key to value).
- Control flow uses if/elif/else and loops (for, while). Indentation defines blocks.
- Functions take parameters and return values. Modules and packages organise code; virtual environments (venv) isolate each project's libraries, and pip installs them.
- Exceptions (try/except) handle errors. Classes bundle data and behaviour into objects with inheritance.

EVERY TOPIC EXPLAINED

Installation and environment setup. Install Python 3 from python.org (Windows: tick 'Add Python to PATH') or with a package manager (brew on macOS, apt on Linux). Check with python --version. Use VS Code with the Python extension. Create a project folder, then a virtual environment (python -m venv .venv) and activate it before installing libraries.

Variables, data types, strings and numbers. A variable is a name for a value: price = 499. Core types: int (whole numbers), float (decimals), str (text in quotes), bool (True/False), None (no value). Strings support slicing (name[0:3]), methods (.upper(), .split(','), .strip()) and f-strings (f'Total: {price}'). Numbers support + − * / // (whole division) % (remainder) ** (power).

Lists, tuples, sets and dictionaries. A list is ordered and changeable: leads = ['Asha', 'Ravi']; leads.append('Meera'). A tuple is ordered and fixed: point = (12.9, 77.6). A set holds unique items and supports union and intersection: {'a','b'} & {'b','c'}. A dictionary maps keys to values: customer = {'name': 'Asha', 'plan': 'pro'}; customer['plan'] reads a value. Dictionaries are the Python shape of JSON.

Operators, conditions and loops. Operators compare and combine values: == != < > <= >=, and, or, not, in. if / elif / else run code only when a condition is true. for loops repeat over each item (for lead in leads:), while loops repeat while a condition holds. break leaves a loop early; continue skips to the next item. range(5) gives 0 to 4.

Functions, parameters and return values. A function is a named, reusable block: def total(price, qty=1): return price * qty. Parameters are the inputs (qty has a default value). return sends a result back. Keyword arguments (total(price=100, qty=3)) make calls readable. Keep functions small with one job; add a docstring explaining what they do.

Modules and packages; file handling; exception handling. A module is one .py file; a package is a folder of modules. import json or from pathlib import Path loads them. File handling: with open('leads.txt', encoding='utf-8') as f: text = f.read() (the with block closes the file). Exceptions handle errors: try: ... except ValueError as e: ... finally: ... so one bad record does not crash the whole job.

Object-oriented programming: classes, objects, inheritance. A class is a blueprint; an object is one instance. class Lead: def __init__(self, name, phone): self.name = name; self.phone = phone; def label(self): return f'{self.name} ({self.phone})'. Inheritance reuses a parent class: class HotLead(Lead) adds or overrides behaviour. Use classes when data and the actions on it belong together.

JSON, CSV and Excel; virtual environments; pip; reusable code. json.load / json.dump read and write JSON; the csv module or pandas.read_csv read CSV; pandas.read_excel (with openpyxl) reads Excel. A virtual environment keeps each project's libraries separate. pip install requests installs a package; pip freeze > requirements.txt records versions so others can rebuild the same environment. Reusable code lives in functions and modules that other scripts import.

WORKED EXAMPLE

> A script reads a CSV of leads, skips rows with a missing email with an if-statement, and writes the valid ones to a JSON file.

RESOURCES

- Python tutorial (official) https://docs.python.org/3/tutorial/
- Python venv docs (official) https://docs.python.org/3/library/venv.html

1.3 JavaScript and TypeScript

DEFINITION

> JavaScript is the language of the browser and of Node.js servers. TypeScript adds static types on top of JavaScript and compiles to it.

HOW IT WORKS

- Async code uses Promises and async/await so a slow network call does not freeze the program.
- JSON is the common data format for APIs; JavaScript reads and writes it natively.
- TypeScript checks types at build time, so mistakes such as passing a number where a string is expected are caught before running.
- Most AI front ends (React, Next.js) are written in TypeScript.

EVERY TOPIC EXPLAINED

JavaScript fundamentals, variables, functions, arrays and objects. JavaScript runs in every browser and on servers with Node.js. let and const declare variables (const cannot be reassigned). Functions: function add(a, b) { return a + b } or arrow form const add = (a, b) ⇒ a + b. Arrays: [1, 2, 3].map(x ⇒ x * 2). Objects: { name: 'Asha', plan: 'pro' } with obj.name access.

ES6 concepts; Promises; async and await; JSON. ES6 added let/const, arrow functions, template strings (`Hi ${name}`), destructuring (const { name } = user), spread (...items), modules (import/export) and classes. A Promise represents a future result. async/await makes asynchronous code read top to bottom: const res = await fetch(url); const data = await res.json(). JSON.parse and JSON.stringify convert between text and objects.

TypeScript types, interfaces and type safety. TypeScript adds types: let price: number; function greet(name: string): string. An interface describes an object's shape: interface Lead { name: string; phone: string; score?: number } (score is optional). The compiler (tsc) reports type errors before the code runs, which prevents a large class of bugs in bigger apps.

JavaScript vs TypeScript; practical API integration. JavaScript is flexible and fast to start; TypeScript is safer and easier to maintain as a project grows, and it is the default in most React and Next.js projects. Practical API integration: a typed function fetches an endpoint, checks res.ok, parses JSON into an interface, handles errors with try/catch and shows a loading state in the UI.

WORKED EXAMPLE

> A React page calls a FastAPI endpoint with fetch(), awaits the JSON reply and shows a typed list of results.

RESOURCES

- TypeScript handbook (official) https://www.typescriptlang.org/docs/
- MDN JavaScript guide https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

1.4 Data Structures and Algorithms

DEFINITION

> Data structures store data in a form that suits the work (arrays, linked lists, stacks, queues, hash tables, trees, graphs). Algorithms are step-by-step procedures that use them. Big O notation describes how the cost grows with input size.

HOW IT WORKS

- An array gives fast access by index; a hash table gives near-constant lookup by key; a queue serves items first in, first out.
- Sorting and searching are the core algorithms. Binary search finds an item in a sorted list in about log n steps.
- Recursion solves a problem by solving smaller copies of it; every recursion needs a base case.
- Big O: O(1) constant, O(log n) logarithmic, O(n) linear, O(n^2) quadratic. You compare options by their growth, not their speed on one machine.

EVERY TOPIC EXPLAINED

Arrays, linked lists, stacks, queues, hash tables, trees and graphs. Array: items in order with fast access by index. Linked list: items point to the next item; fast insert, slow lookup. Stack: last in, first out (undo history). Queue: first in, first out (job processing). Hash table (Python dict): key to value with near-constant lookup. Tree: hierarchy (folders, org charts); a binary search tree keeps items sorted. Graph: nodes and edges (road maps, social networks, knowledge graphs).

Searching, sorting and recursion. Linear search checks each item (O(n)); binary search halves a sorted list each step (O(log n)). Sorting algorithms include merge sort and quicksort (O(n log n)); in practice use the built-in sorted(). Recursion: a function calls itself on a smaller input until a base case, for example walking a folder tree or computing factorial(n) = n × factorial(n − 1).

Big O notation, time and space complexity, problem decomposition. Big O describes how time or memory grows as input grows: O(1) constant, O(log n), O(n), O(n log n), $O(n^2)$. Space complexity is the extra memory used. Problem decomposition breaks a big problem into small steps you can solve and test one by one: understand inputs and outputs, list cases, solve the simplest case, then generalise.

WORKED EXAMPLE

> Looking up a customer by ID in a dict takes about the same time for 100 or 1,000,000 customers, while scanning a list grows with the list size.

RESOURCES

- Python data structures (official) https://docs.python.org/3/tutorial/datastructures.html
- VisuAlgo (visual algorithms) https://visualgo.net/en

1.5 Git and GitHub

DEFINITION

> Git records the history of a project as commits. GitHub hosts Git repositories online and adds pull requests, issues and reviews.

HOW IT WORKS

- A repository is a folder tracked by Git. You stage changes (git add), save them as a commit (git commit), and push them to GitHub.
- Branches let you work on a change in isolation. A pull request asks to merge your branch into the main branch after review.
- Merge conflicts happen when two branches change the same lines; you resolve them by editing the file and committing.
- A README explains the project; issues track tasks and bugs.

EVERY TOPIC EXPLAINED

Repositories, cloning, committing, branching and merging. git init creates a repository; git clone URL copies one from GitHub. git status shows changes; git add stages them; git commit -m 'message' saves a snapshot. git branch feature-x and git switch feature-x create and move to a branch. git merge combines branches. git push sends commits to GitHub; git pull brings others' commits down.

Pull requests, conflict resolution, README files. A pull request (PR) proposes merging a branch, shows the diff and collects review comments and CI results. A merge conflict happens when two branches edit the same lines; Git marks the conflict with <<<<<<< ======= >>>>>>>, you keep the right lines, remove the markers, then add and commit. A README.md explains what the project does, how to install and run it, and how to use it.

Issues, projects and portfolio development. GitHub Issues track bugs and tasks with labels and assignees; GitHub Projects arranges them on boards. For a portfolio, each project repository needs a clear README with the problem, a screenshot or demo link, the architecture, how to run it and what you learned. Recruiters and clients read READMEs first.

WORKED EXAMPLE

> You create a branch feature/fde-demo, commit a demo app, open a pull request, and merge it after a review.

RESOURCES

- Git documentation (official) https://git-scm.com/doc
- GitHub Docs: pull requests https://docs.github.com/en/pull-requests

1.6 Software Engineering

DEFINITION

> Software engineering is the discipline of building software reliably with a process, design, testing and maintenance over its whole life.

HOW IT WORKS

- The SDLC moves through requirements, design, build, test, release and maintain. Agile works in short iterations; Scrum uses sprints and roles; Kanban limits work in progress on a board.
- Clean code uses clear names, small functions and one responsibility per module.
- Testing: unit tests check one function; integration tests check parts together; code review catches defects and spreads knowledge.
- Design patterns (factory, strategy, observer) are reusable solutions. SOLID principles keep classes focused, extensible and easy to change.

EVERY TOPIC EXPLAINED

Software Development Life Cycle; Agile; Scrum and Kanban. The Software Development Life Cycle (SDLC) runs requirements → design → implementation → testing → deployment → maintenance. Agile delivers in small increments with frequent feedback. Scrum uses fixed sprints (1–4 weeks), roles (product owner, scrum master, developers) and events (planning, daily stand-up, review, retrospective). Kanban visualises work on a board and limits work in progress.

Requirements analysis, software design, modular programming, clean code. Requirements analysis turns needs into clear, testable statements. Software design chooses the components, data and interfaces. Modular programming splits code into independent parts. Clean code uses meaningful names, short functions, no duplication and comments that explain why, not what.

Code reviews, testing fundamentals, debugging, logging, documentation. Code review: a teammate reads your change before merge to catch bugs and share knowledge. Testing: unit tests (pytest) check small parts; integration tests check parts together. Debugging finds the cause of a bug using error messages, print/logging and a debugger with breakpoints. Logging records what happened in production. Documentation explains how to use and maintain the code.

Design patterns, SOLID principles, refactoring, maintenance. Design patterns are proven solutions: Factory (create objects without naming the exact class), Strategy (swap an algorithm, such as which AI model to call), Observer (notify listeners of events), Adapter (wrap an incompatible API). SOLID: Single responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion. Refactoring improves structure without changing behaviour, protected by tests. Maintenance fixes bugs and updates dependencies over time.

WORKED EXAMPLE

> A team adds a discount feature: a requirement is written, the logic is a pure function with unit tests, a reviewer checks it, and it is released behind a feature flag.

RESOURCES

- Scrum Guide (official) https://scrumguides.org/scrum-guide.html

- Python testing with pytest https://docs.pytest.org/en/stable/

1.7 Development Tools

DEFINITION

> Development tools are the editors, terminals, package managers, debuggers and AI assistants that let you write, run and check code.

HOW IT WORKS

- The terminal runs commands; package managers (pip, npm) install libraries; the debugger steps through code line by line.
- AI coding assistants (GitHub Copilot, Cursor, Claude Code) suggest or write code. You must read, run and test the output: AI code can be wrong or insecure.
- Use them to explain, draft and refactor; verify with tests and review before merging.

EVERY TOPIC EXPLAINED

VS Code, Cursor, GitHub, GitHub Copilot, Claude Code. VS Code is a free, extensible editor. Cursor is an AI-first fork of VS Code with chat and multi-file edits. GitHub hosts code and runs CI. GitHub Copilot suggests code inline. Claude Code is an AI coding agent in the terminal and IDE that reads your repository, edits files and runs commands with your permission.

Terminal and command line; package managers; debugging tools. The terminal runs commands: cd (change folder), ls/dir (list), mkdir (make folder), python app.py (run), git status. Package managers install libraries: pip (Python), npm (JavaScript), brew/apt (system). Debugging tools: breakpoints, step over/into, variable watch, browser DevTools (Console, Network, Elements).

Using AI coding assistants responsibly without accepting code blindly. AI assistants speed up drafting, explaining and refactoring, but they can produce code that looks right and is wrong, insecure or outdated. Always read the code, run it, write or run tests, check for hard-coded secrets and unsafe input handling, and understand every line you commit. Never paste customer secrets or personal data into tools your company has not approved.

WORKED EXAMPLE

> You ask Claude Code to write a function, run the test suite, read the failing test, and fix the function yourself before committing.

RESOURCES

- VS Code documentation https://code.visualstudio.com/docs
- GitHub Copilot docs https://docs.github.com/en/copilot